

Introduction

The world is progressing towards digitalization, and the transportation area is no exception. One of the most significant obstacles people face in urban areas, particularly in large cities, is finding parking.

From the last few years, the number of vehicles is increasing day by day on the roads but the number of roads and parking area have not increased proportionally due to which there is a severe parking shortage in urban areas. Now we all are facing the new challenge of vehicle parking. It is observed that more than one million vehicles consume oil on daily basis in wait of finding the parking slot, that cause wastage of oil and emission of smoke/carbon in air that increases air pollution. To overcome these problems E-Parking is the solution that is proposed.

The project will require a comprehensive analysis of the current parking system, including identifying the key pain points and challenges faced by drivers, parking operators, and city authorities. The system will be designed to integrate with existing parking management infrastructure and will use modern technologies of IOT to replace conventional parking system.

In the proposed system instead of waiting the user will use android app to check the availability of parking slots. Then RFID is used to authenticate the user and once the user is verified the required amount is deducted and user can park on the available slots shown on App.

1.1 Goals

"Our goal is to design a real-time Android Based E-Parking System based on IoT that will automate the parking system using automation technology and reduce the human effort and time by providing seamless and efficient parking experience to users".

1.2 Motivation

We have developed Android Based E-Parking system so that we can

overcome the emerging problems of parking that are increasing day by day. What motivated us to develop this system is its non-presence in our country, even in the International Airports of Pakistan, societies and malls we all had faced the problem of finding parking slot because of conventional parking system. There were also some other reasons for choosing this project, for example, the rate of air pollution is increasing day by day and one of the major contributions to this pollution is emission of carbon/smoke from vehicles, and thousands of vehicles consumes oil daily in wait of finding the parking slot. So, this system can help to reduce carbon emission and contribute towards mitigating the effects of climate change.

1.3 Methods

This project is developed using incremental model because in this process of software development requirements are broken down into multiple modules. It is iterative and incremental approach to software development where project is broken down into smaller and manageable increments or modules, which means each module involves designing, coding, testing and implementing a specific feature or functionality. Using incremental model, we can develop this project in following increments:

Requirement Gathering: For the E-Parking System gather the initial requirements and functionalities that may include understanding main functionality of system, user interface designs and other requirements.

Design Phase: Create a design of system architecture that include Android application design and user interface design, simulation of hardware connections with RFID integration approach.

Increment 1: Developing the basic foundation of the system that includes creating Android application framework, creating frontend or user interface of the application.

Increment 2: Creating a simulation of working of hardware used for the prototype. That shows the complete working of hardware and connecting the hardware components to witness it's working.

Increment 3: Coding backend of hardware and setting up server and database

for the system to store user information.

Increment 4: After setting up all the things integrate hardware and software and establish a connection between hardware and software so that the data can be transferred, for example, showing available and occupied slots on App.

Increment N: Based on the feedbacks from concerned authorities adding extra features and functionalities such as adding duration option that shows for how much time the vehicle is parked and maintaining the system.

By Using incremental model, the project is developed iteratively and each increment adds a new feature and functionality, ensuring a proper working and manageable development process that allows changing and adding of any feature at any time.

1.4 Report Overview

This report has introduced a system that allows a reader to explore the initial concepts of the system. The report consists of eight chapters.

Chapter 1 reviews about the introduction and basic motives of the system. It is the overview the whole system and about the factors involving in developing the system. Main ideas and abstraction of the system are carried out in this chapter.

Chapter 2 reviews the background of the project. Problem statements which we found to develop this system. This section will review literature that will provide the understanding and explanation of critical success factors in project management and discussion system. The literature review will include Project success factors, Critical success factors and social interaction among the users.

Chapter 3 reviews the plan through which the project is carried out. Methods and approach used to develop the project are discussed in the section. Gantt charts are made to show the work plan in which the system is developed.

Chapter 4 reviews the requirements of the system. Functional and non-functional requirements are discussed. Use cases for different users are discussed and their functions are discussed.

Chapter 5 and 6 reviews the design and architecture of the system. Features that the product have been discussed and explored in the section. User interfaces are shown in the section. System architectures and designs are designed in this section.

Chapter 7 reviews the coding methods and techniques and testing of the product. Test cases are designed to check the product.

Chapter 8 reviews the overall system and different key point observed in developing the system. References are discussed using which we got help.

CHAPTER 2

Background and Problem Statement

The non-presence of a smart parking system is one of the major causes of parking crises in urban areas. As we know that the number of vehicles is increasing day by day so the management needs E-Parking system to manage the parking issues. In current conventional parking system finding a parking spot can be a time consuming and frustrating job which is why E-Parking System can be a game changer for parking management.

Another problem is that due to non-presence of E-Parking System and terrible conventional parking system people park their vehicles anywhere they want that causes traffic jams and other people suffer due to wrong parking of one person.

2.1 Introduction

With Android Based E-Parking System instead of waiting for finding parking slots a user can see all the available parking slots on its android device that can be mobiles or tablets or the android screen embedded in the cars. Then using the card or tag user can access the parking area and park their cars on the available slots by saving their time and fuel that can be wasted in search of parking slot. E-Parking System is completely automated system which means zero human intervention, all the barriers (entry and exit) automatically opens and close and using RFID technology and integration of payment gateways we don't need any human to check the identity or to issue any type of ticket that is used in conventional parking systems.

2.2 Literature Review

Automated Parking Systems are becoming popular throughout the world because of ease and functionalities provided by them. Due to these systems parking managements save their time and resources due to zero human intervention.

In the 1980s, one system built in Honolulu was a part of a small office construction on a crowded mixed-use street. Then, in the early 80's, automated parking solution were embraced by Japan and USA and spread to Australia, Southeast Asia, China, Philippines and Singapore. Between early 70's and late 80's, around more than hundred automated systems were built worldwide.

Many researchers discuss various methods to replace conventional parking system to E-Parking Systems. Some researchers discuss GSM technology to automate this system where user send message to GSM modem that is placed at parking zone. Then the GSM modem send the reply to user or sender whether the parking slot is available or not.

Some other researchers discuss a system that uses some digital key along with some robotic techniques. In this method when any car enters the driver has to enter a valid key and choose option of parking or retrieving. Upon entering the valid key, the car is picked up along with the pallet and placed in stack system and once the driver returns and enter valid key the system will check and car return back to drive way. Here the stack system pulls down the pallets to make room for incoming pallets. This system includes robotic lift with motors for picking the car and placing it in designating spots. But this was time consuming system.

After that some researchers discuss this automated system using RFID technology to automate conventional parking systems. In this system user have to register first and get the RFID tag or card. As the RFID card is read by the RFID reader, the system automatically deducts the specified amount and the entry barrier opens. The available parking slots are shown in the app and user can park on it. Similarly, the exit barrier opens automatically when vehicle is detected near the barrier.

2.3 Problem Statement

The non-presence of a smart parking system is one of the major causes of parking crises in urban areas. The number of automobiles on the road has significantly increased. However, the number of roads and parking lots has not increased in a way that is proportional to the number of vehicles, which is unfortunate. Due to this, there is a severe parking shortage, particularly in urban areas. Due to this people park their vehicle where ever they want specially on roads that creates a lot of problems of traffic jam and the common man suffer it every day.

CHAPTER 3

Project Management

This project is developed using incremental model because in this process of software development requirements are broken down into multiple modules. It is iterative and incremental approach to software development where project is broken down into smaller and manageable increments or modules, which means each module involves designing, coding, testing and implementing a specific feature or functionality.

3.1 Approach

Incremental approach is a approach in which software development requirements are broken down into smaller and manageable increments or modules. Hence, each increment involves its own designing, developing, testing and implementig so the risk of errors is reduced.

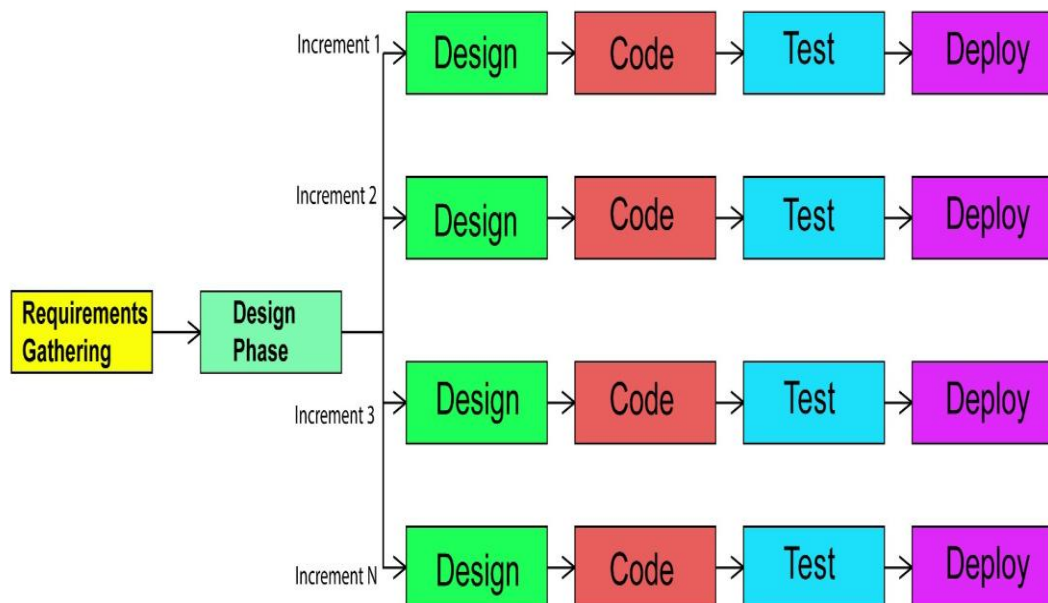


Figure 3.1: Incremental Model

3.2 Initial Project Plan

ID	 Task Mode	Task Name	Duration	Start	Finish
0		Android Based E-Parking System	138 days	03/01/2024	8/8/2024
1		Requirement Gatering	6 days	03/01/2024	03/07/2024
2		Determine project scope	1 day	03/01/2024	03/01/2024
3		Gather Initial Requirements	5 days	03/02/2024	3/7/24
4		Collect Literature	18 days	3/8/24	3/28/24
5		Collect and Study Literature	10 days	3/8/24	3/18/24
6		Validate it	8 days	3/20/24	3/28/24
7		Hardware Selection	6 days	3/29/24	4/4/24
8		Select Hardware for Prototype	6 days	3/29/24	4/4/24
9		Proposed Methodology	6 days	4/5/24	4/11/24
10		Identify Approach for Project	2 days	4/5/24	4/6/24

Figure 3.2: Project schedule plan

ID	 Task Mode	Task Name	Duration	Start	Finish
11		Proposed Methodology for Project	4 days	4/7/24	4/11/24
12		System Design	24 days	4/12/24	5/9/24
13		Prototype Design	10 days	4/12/24	4/22/24
14		Architecture Design	14 days	4/24/24	5/9/24
15		Installing Hardware	6 days	5/10/24	5/16/24
16		Making Prototype	6 days	5/10/24	5/16/24
17		Development	48 days	5/17/24	7/11/24
18		Coding	30 days	5/17/24	6/20/24
19		Database Design	8 days	6/21/24	6/29/24
20		Implementation	4 days	6/30/24	7/4/24
21		Testing	6 days	7/5/24	7/11/24
22		Deployment	24 days	7/12/24	8/8/24
23		Deployment	3 days	7/12/24	7/14/24
24		Documentation	21 days	7/15/24	8/8/24

Figure 3.2.1: Project schedule plan

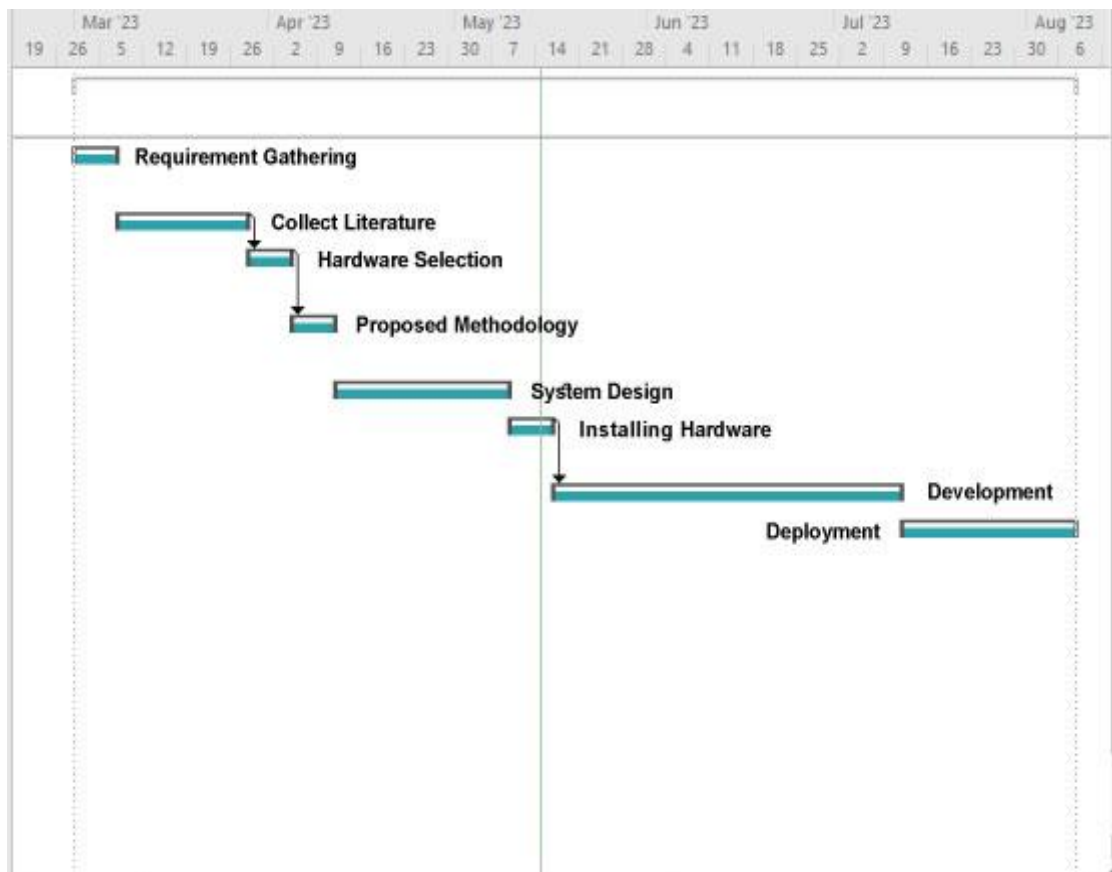


Figure 3.3: Project Gantt chart

3.3 Problems and Change to the Plan

We did not face any major problems during the development of this project but if any organization would like to use this type of project in real time, we need to customize it according to their more specific needs. But for this FYP project we worked according to our provided requirements. Our Android app and prototype is completely in working condition.

CHAPTER 4

Analysis

The main focus of this project is to provide safe and convenient automated parking system to users so that they can save their time by finding available parking slots on the screen of their android devices.

4.1 Use cases

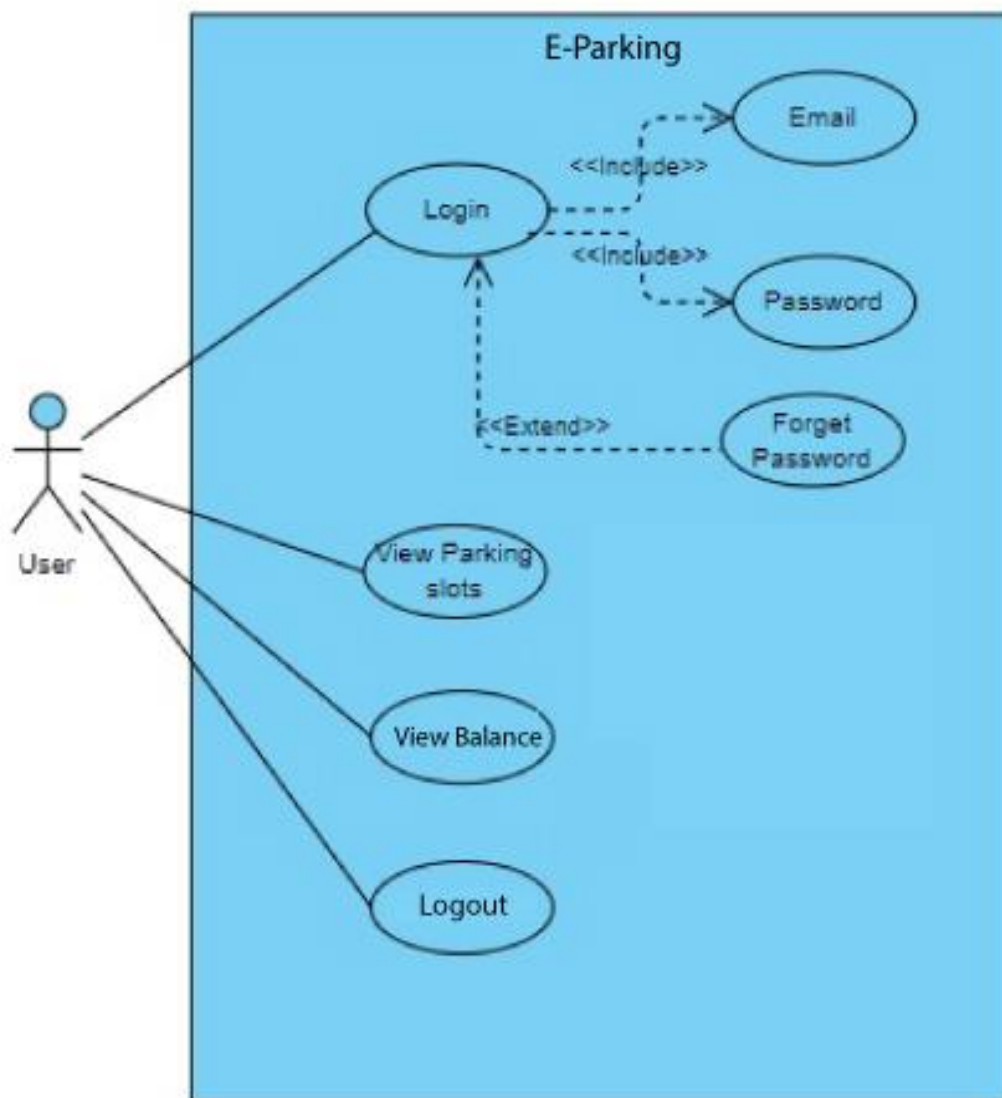


Figure 4.1: User Use case

Table 4.1: Use-case Login

Use Case1: Login
ID: UC-01
Actor: User
Pre-condition: User must have a profile
The flow of events: - User will enter Email and Password. - User will Click on login Button. - The Application will allow the user to use all the feature of the Application.
Secondary scenarios: User can go to reset password.
Post condition: App transfer control to user main screen to proceed further actions

Table 4.2: Use-case Forget Password

Use Case2: Forget Password
ID: UC-02
Actor: User
Pre-condition: User must have a profile
The flow of events: - User will contact Admin - User will check email for reset - After successful verification the password will be reset.
Secondary scenarios: User can go to login page.
Post condition: App transfer control to user main screen to proceed further actions

Table 4.3: Use-case View Parking Slots

Use Case3: View Parking Slots
ID: UC-03
Actor: User
Pre-condition: User must be login.
The flow of events: - User will login into the system. - User can check all the parking slots (available and occupied). - User can park on any available parking slot.
Secondary scenarios: None.
Post condition: Amount will be deducted.

Table 4.4: Use-case View Wallet

Use Case4: View Balance
ID: UC-04
Actor: User
Pre-condition: User must be login.
The flow of events: - User will login into the system. - User can check its remaining Balance
Secondary scenarios: User can come back to main screen.
Post condition: App transfer control to user main screen to proceed further actions.

Table 4.5: Use-case Add Money

Use Case5: Add Money
ID: UC-05
Actor: User
Pre-condition: User must be login.
The flow of events: - User will login into the system. - User can check its wallet - User can deposit money for parking.
Secondary scenarios: User can come back to main screen.
Post condition: App transfer control to user main screen to proceed further actions.

Table 4.6: Use-case Add Money

Use Case6: User Logout
ID: UC-06
Actor: User
Pre-condition: User must be login.
The flow of events: - User will press logout button. - User will end its session.
Secondary scenarios: User can come back to main screen.
Post condition: None

4.1.2 Admin use cases

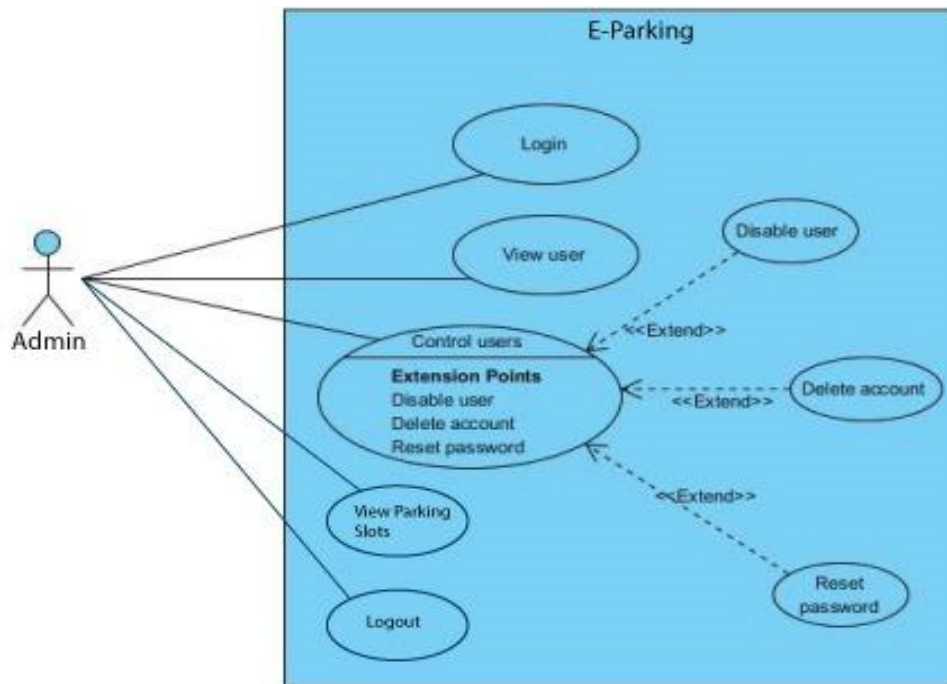


Figure 4.2: Administrator Use case

Table 4.7: Use Case of Login

Use Case7: Login
ID: UC-07
Actor: Administrator
Pre-condition: Administrator must be registered
Flow of events: - Admin will enter Email and Password. - Admin will Click on login Button. - The Application will allow the admin to use all the feature of the Application.
Secondary scenarios: None
Post condition: Administrator can control the users

Table 4.8: Use Case of View Users

Use Case8: View users
ID: UC-08
Actor: Administrator
Pre-condition: Administrator must be logged in
Flow of events: - Administrator can login to the system - In the authentication section administrator can view the users - Here administrator can view the user, add user, delete and reset password.
Secondary scenarios: None
Post condition: Administrator can control the users

Table 4.9: Use Case of Add Users

Use Case9: Add Users
ID: UC-09
Actor: Administrator
Pre-condition: Administrator must be logged in
Flow of events: - Administrator can Add Users to the system - In the authentication section administrator can view the users - Here administrator can view the user, add user, delete and reset password.
Secondary scenarios: Admin can view empty slots
Post condition: Administrator can control the users

Table 4.10: Use Case of Delete Users

Use Case10: Delete Users
ID: UC-10
Actor: Administrator
Pre-condition: Administrator must be logged in
Flow of events: - Administrator can Delete Users from the system - In the authentication section administrator can view the users - Here administrator can delete the users
Secondary scenarios: Admin can add new user
Post condition: Administrator can control the users

Table 4.11: Use Case of Reset User Password

Use Case11: Reset Password
ID: UC-11
Actor: Administrator
Pre-condition: Administrator must be logged in
Flow of events: - Administrator can send reset link to users - In the authentication section administrator can view the users
Secondary scenarios: Admin can add new user
Post condition: Administrator can control the users

Table 4.12: Use Case of Disable User

Use Case12: Disable Users
ID: UC-12
Actor: Administrator
Pre-condition: Administrator must be logged in
Flow of events: - Administrator can disable users - In the authentication section administrator can disable the users for any violation
Secondary scenarios: Admin can add new user
Post condition: Administrator can control the users

Table 4.13: Use Case of Logout

Use Case13: Admin Logout
ID: UC-13
Actor: Administrator
Pre-condition: Administrator must be logged in
Flow of events: - Administrator can Logout of the system - After logout the admin can login again for new session
Secondary scenarios: None
Post condition: None

4.2 Functional Requirements

Functional requirements are the specific behaviors, tasks, or functions a system must perform to meet its intended purpose. These requirements define the actions or operations the system is expected to carry out in response to inputs, interactions, or specific conditions. They focus on what the system should do, outlining features, capabilities, and business rules that ensure the system operates as intended.

Table 4.14: Functional Requirements

Requirement #	Requirement
REQ-1	The app shall provide a login and logout functionality for all users
REQ-2	New user must be added by Admin and receive RFID card/tag.
REQ-3	User must be able to view all parking slots (available and occupied).
REQ-4	User must be able to view the wallet balance
REQ-5	User can deposit money in his wallet that will be deducted for accessing parking area.
REQ-6	User can reset password by Admin
REQ-7	User must have RFID card to access parking area.
REQ-8	Admin can register or add new users.
REQ-9	Admin can delete any user.
REQ-10	Admin can reset any user's password.
REQ-11	Authorization is checked using RFID Reader.
REQ-12	Parking zones barrier must be automated.

4.3 Non-Functional Requirements

Table 4.15: Non- Functional Requirements

Requirement #	Requirement
REQ-1	Maintainability
REQ-2	Portability
REQ-3	Reliability
REQ-4	Supportability
REQ-5	Availability

CHAPTER 5

Product/System Design

5.1 Introduction

The system is developed using the SE software design practices. Different UML diagrams are designed to make system structure and working clear. We used use case diagram which helped in identifying the user actions and functions. We designed sequence diagram to check the sequence in which the actions are performed. Schema diagram is used to give the concept of how data will be stored in Firebase Tree structure. Activity diagram to understand the activities being performed by the users and auctioneer in the system. The class diagram is designed to illustrate the relationships between the classes.

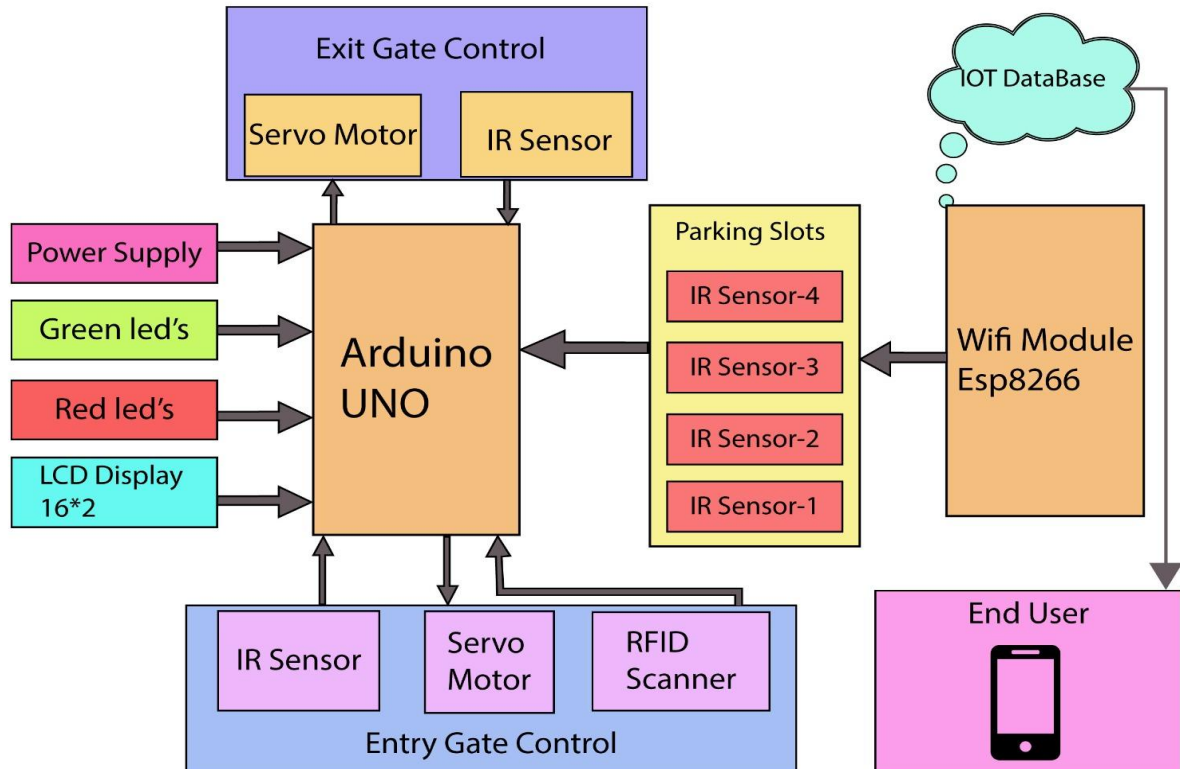


Figure 5.1: Application Architecture diagram

5.2 Product Features

As a whole project has different features in order to interact with humans. The system is proposed to have features like login, view parking slots, RFID access, automated opening and close barriers.

To interact with App user will login the App to interact with it using email and password of the particular user.

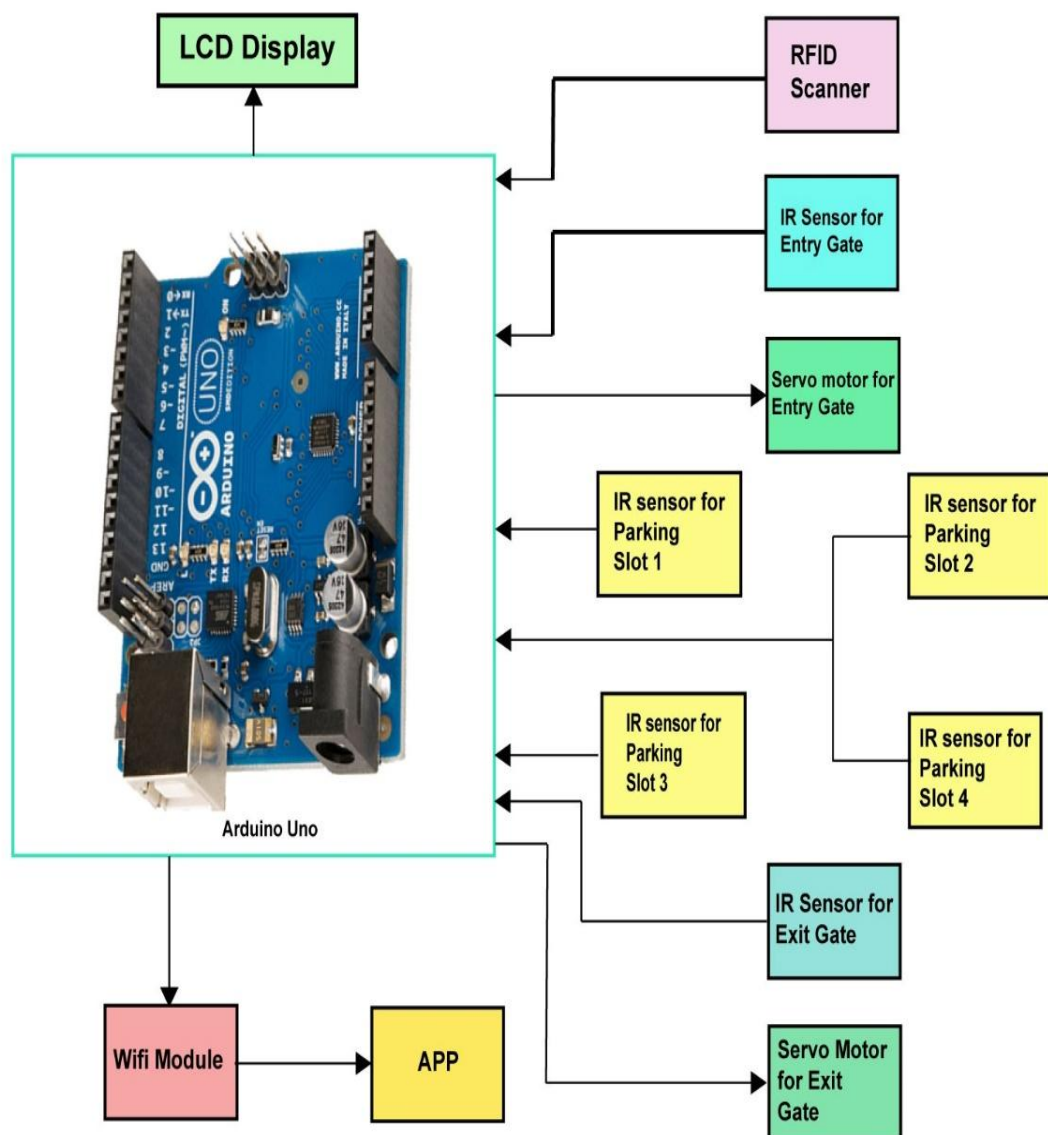


Figure 5.2: Project Working

5.3 User Interface

User interfaces include the following UI elements:

- Buttons
- Labels
- Edit text
- Touchable icons
- Pop-Up Notification
- Alerts
- Loading-Bar
- Placeholder
- Text View

User interfaces (UIs) are designed to facilitate seamless interaction between users and applications, incorporating a variety of essential elements. Buttons enable users to perform actions with a simple tap or click, while labels provide static text for guidance or description. Editable text fields (Edit Text) allow users to input data such as usernames or passwords, complemented by placeholders to hint at the required input format. Touchable icons offer interactive options like search or share functionality, enhancing usability. Pop-up notifications deliver brief, non-intrusive updates, whereas alerts present critical information or decision points requiring user acknowledgment. Loading bars visually indicate progress, ensuring users are informed of ongoing processes, while text views display static or dynamic content in an application. Together, these elements create intuitive and efficient user experiences.

In addition to these fundamental components, dropdown menus allow users to select from multiple options within a compact interface, conserving screen space while providing flexibility. Switches and toggle buttons enable users to quickly activate or deactivate specific settings, offering a more interactive experience. Sliders are useful for adjusting values, such as volume or brightness, with precision through a sliding mechanism. Carousels or galleries display a series of images or content that users can

browse interactively. Modal dialogs temporarily halt interactions on the primary interface, focusing attention on critical actions or messages. Gesture controls, such as swiping, pinching, or tapping, introduce natural and intuitive interaction methods. Tooltips provide on-hover or on-focus explanations, making complex interfaces more user-friendly. Progress circles complement loading bars by providing a visual cue of ongoing tasks in a dynamic way.

Together, these elements not only create intuitive and efficient user experiences but also ensure flexibility, accessibility, and engagement across diverse applications and devices. A well-designed UI integrates these features cohesively, adapting to user preferences and enhancing overall functionality while maintaining aesthetic appeal.



Figure 5.3: UI Login



Figure 5.4: UI Parking Slots Admin

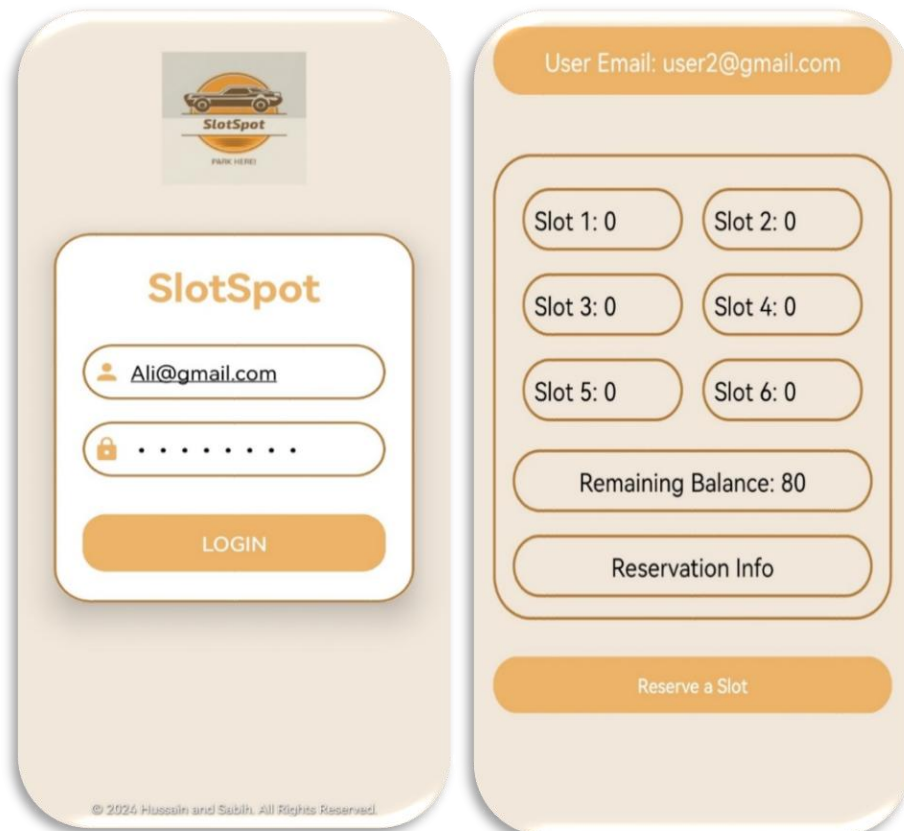


Figure 5.5: UI Parking slots User

5.4 Non-Functional Requirements

Firebase is used as backend server. Firebase provides secure channels of data flow between App, IOT devices and Server. Firebase is providing full support of non-functional requirements:

5.4.1 Performance:

The system should respond to user commands within a reasonable time frame. The IOT devices (Sensors) should be able to send commands to the App quickly, and the App should respond to the commands in a timely manner.

5.4.2 Safety& security requirements:

The system should ensure the security and confidentiality of user data. The system will use Firebase for user authentication and database management, which should provide secure and encrypted data transmission.

5.4.3 Software Quality attributes:

Reliability:

The system should be reliable, and it should work as expected without any failures or crashes. The system should also handle unexpected situations, such as network outages or server downtime, gracefully.

Availability

The system should have strong security measures in place to prevent unauthorized access to the Parking and the mobile application. Authentication and encryption protocols should be implemented to protect user data and prevent any malicious attacks.

Maintainability

The system should be designed with ease of maintenance in mind, allowing for quick and efficient updates and repairs as needed. The system should also have

appropriate documentation to facilitate maintenance and troubleshooting.

5.5 Data Storage

Firebase non-SQL database is used as backend server database, it works on publisher and subscriber model. The prototype used this for the communication between App and Sensors and authentication of users.

Software Design

E-Parking is a native Android application that follows pure object-oriented programming practices. The entire app is designed using various UML diagrams, including use-case, sequence, class, and activity diagrams. As a native Android app, E-Parking is built using Java programming language, and utilizes the Android SDK and Android Studio development environment. The app is optimized for performance and functionality specific to the Android platform.

6.1 Introduction

E-Parking is designed using the Incremental development model, which emphasizes collaboration and flexibility throughout the development process. In the Incremental model, the development process is iterative and incremental, with each phase building on the previous one. This allows for continuous feedback, resulting in a more flexible and responsive development process.

The Incremental development model divides the project into manageable chunks or modules, which are developed and tested individually. Each module goes through requirements analysis, design, implementation, and testing before being integrated with the existing system. This approach ensures that high-priority features are delivered early, allowing stakeholders to use and evaluate the system progressively. As new modules are added, feedback from stakeholders is incorporated into future iterations, ensuring that the final product aligns with user needs and expectations.

One of the significant advantages of the Incremental model is its adaptability to changing requirements. As E-Parking progresses through development, unanticipated challenges or new demands can be addressed without overhauling the entire system. This flexibility ensures that the project remains aligned with evolving goals. Additionally, the modular nature of development helps reduce risks, as each increment is a self-contained deliverable that can be thoroughly tested before

integration. This minimizes errors and ensures that the system is stable and reliable at every stage.

The collaboration aspect of the Incremental model enhances communication between developers, designers, and stakeholders. Regular reviews and feedback sessions foster a sense of shared ownership and ensure that the system meets its intended purpose. Moreover, the early delivery of core functionalities allows users to familiarize themselves with the system, providing valuable insights for refining future increments. This user-centric approach is especially beneficial for projects like E-Parking, where real-time functionality and user convenience are critical to success.

6.2 High-Level Design Use Case

A use case diagram is a powerful graphical tool used to depict the interactions between a system and its users, often referred to as actors. It provides a comprehensive and high-level overview of the system's functionality by illustrating how various types of users engage with different components or features within the system. Each actor in the diagram represents a specific role or user type, which may include individuals, external systems, or devices interacting with the system. The diagram identifies the actions, or use cases, these actors can perform, helping to outline the functional requirements of the system in a way that is easy to understand for both technical and non-technical stakeholders. By visually organizing these interactions, a use case diagram simplifies complex processes, making them more accessible and actionable for analysis and design. This clarity helps stakeholders better understand the system's scope, fosters alignment between teams, and ensures that critical functionalities are accounted for during development. Additionally, use case diagrams are invaluable for identifying potential challenges, streamlining workflows, and ensuring user needs are prioritized. They serve as a bridge between technical experts, such as developers and designers, and non-technical contributors, such as clients and managers, facilitating communication and collaboration throughout the project lifecycle. As a result, use case diagrams play an essential role in system analysis, design, and documentation, enabling teams to work efficiently and produce solutions that effectively meet the desired requirements and goals.

6.3 User Use Case

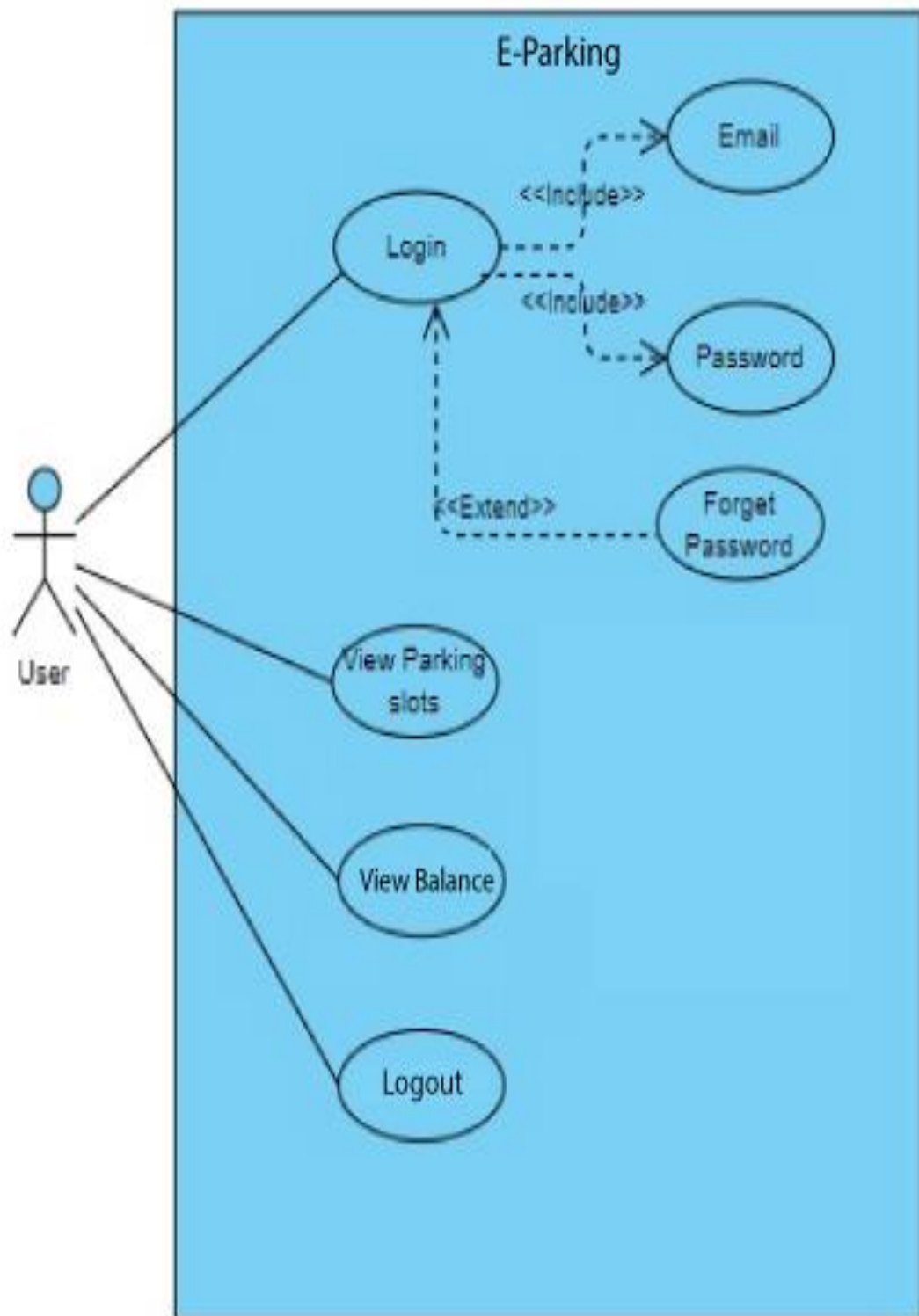


Figure: User Use-cases

Admin Use Case

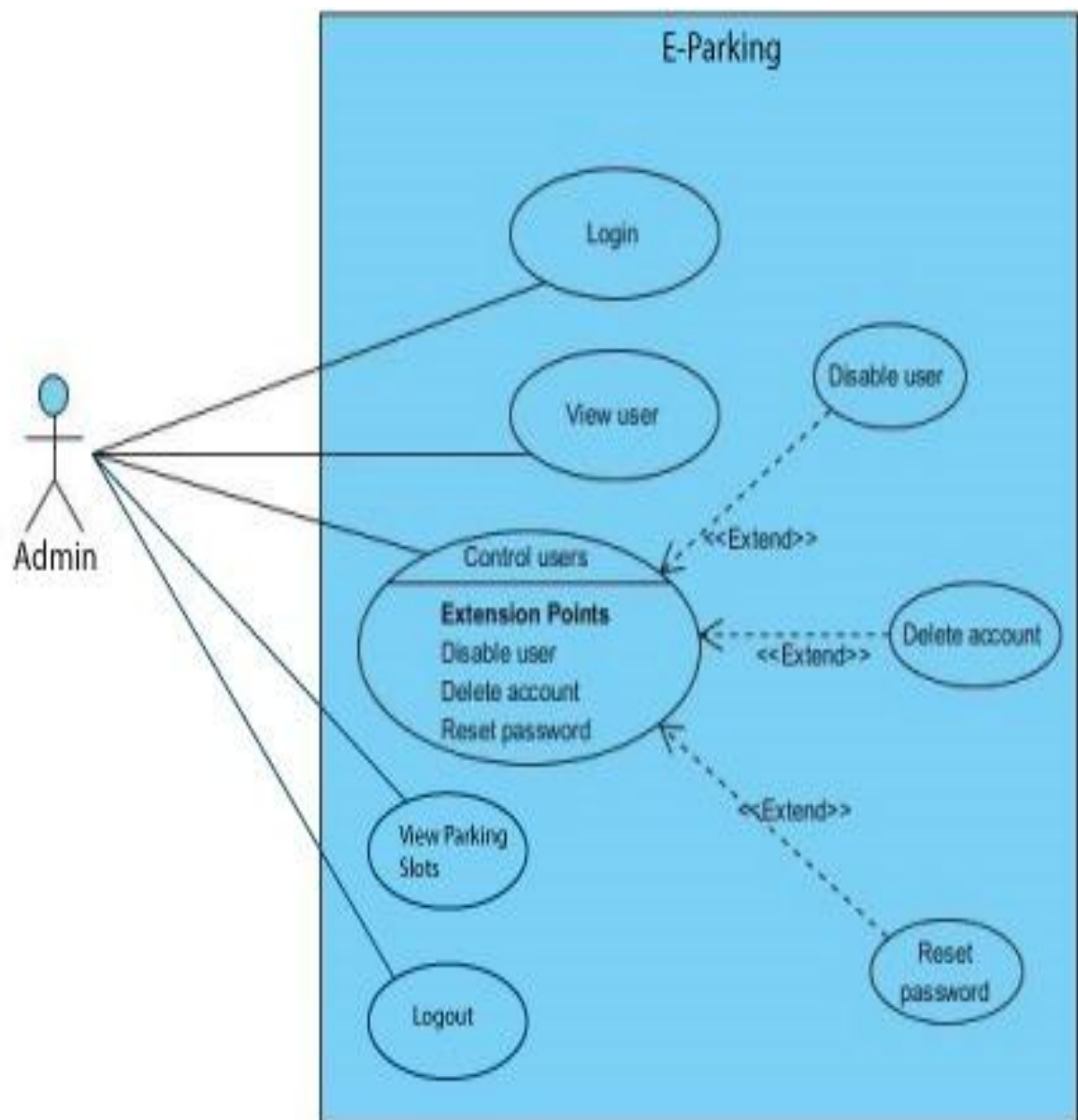


Figure: Admin Use-cases

6.4 Sequence Diagram

A sequence diagram is a type of interaction diagram because it describes how—and in what order—a group of objects works together. These diagrams are

used by software developers and business professionals to understand requirements for a new system or to document an existing process.

User Sequence Diagram

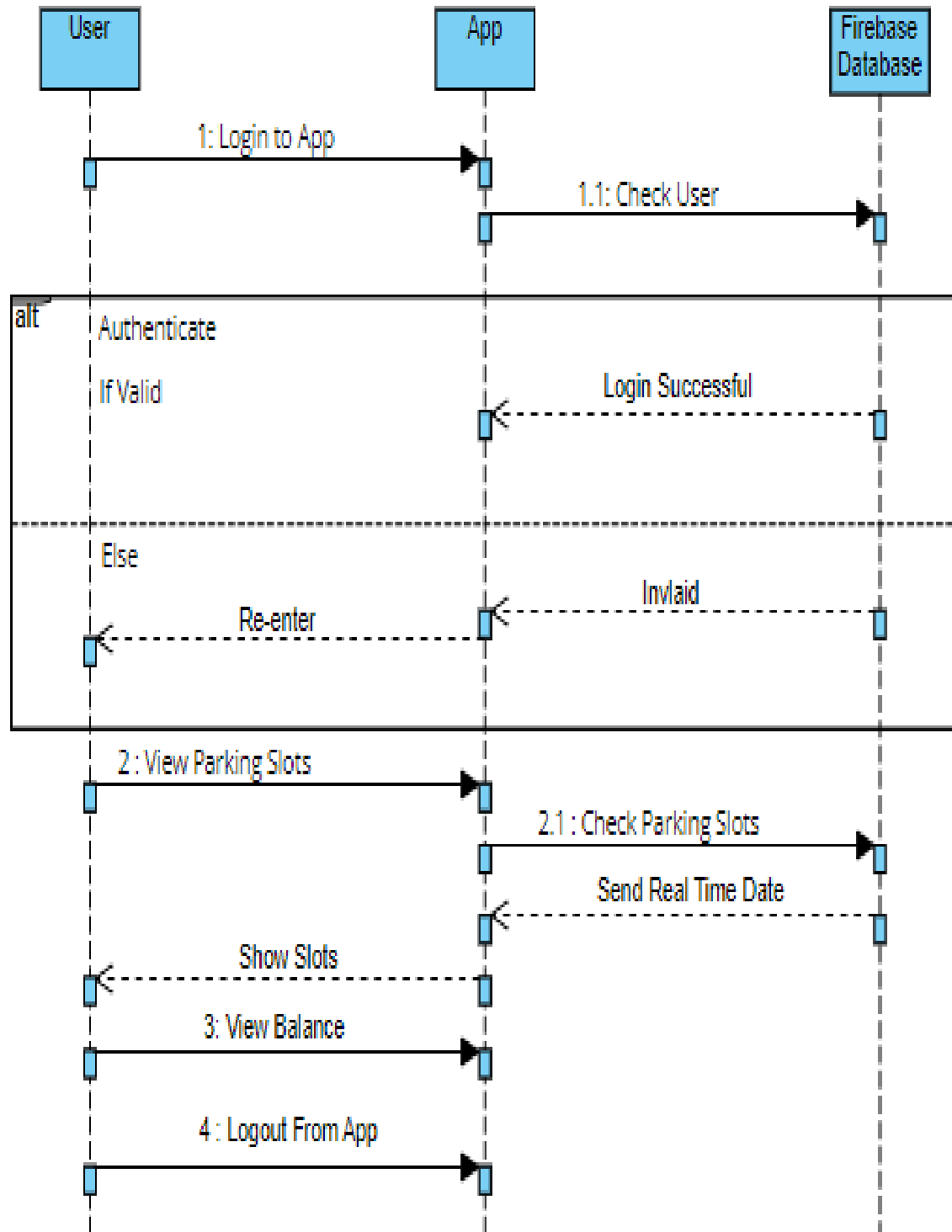


Figure: User Sequence Diagram

Admin Sequence Diagram

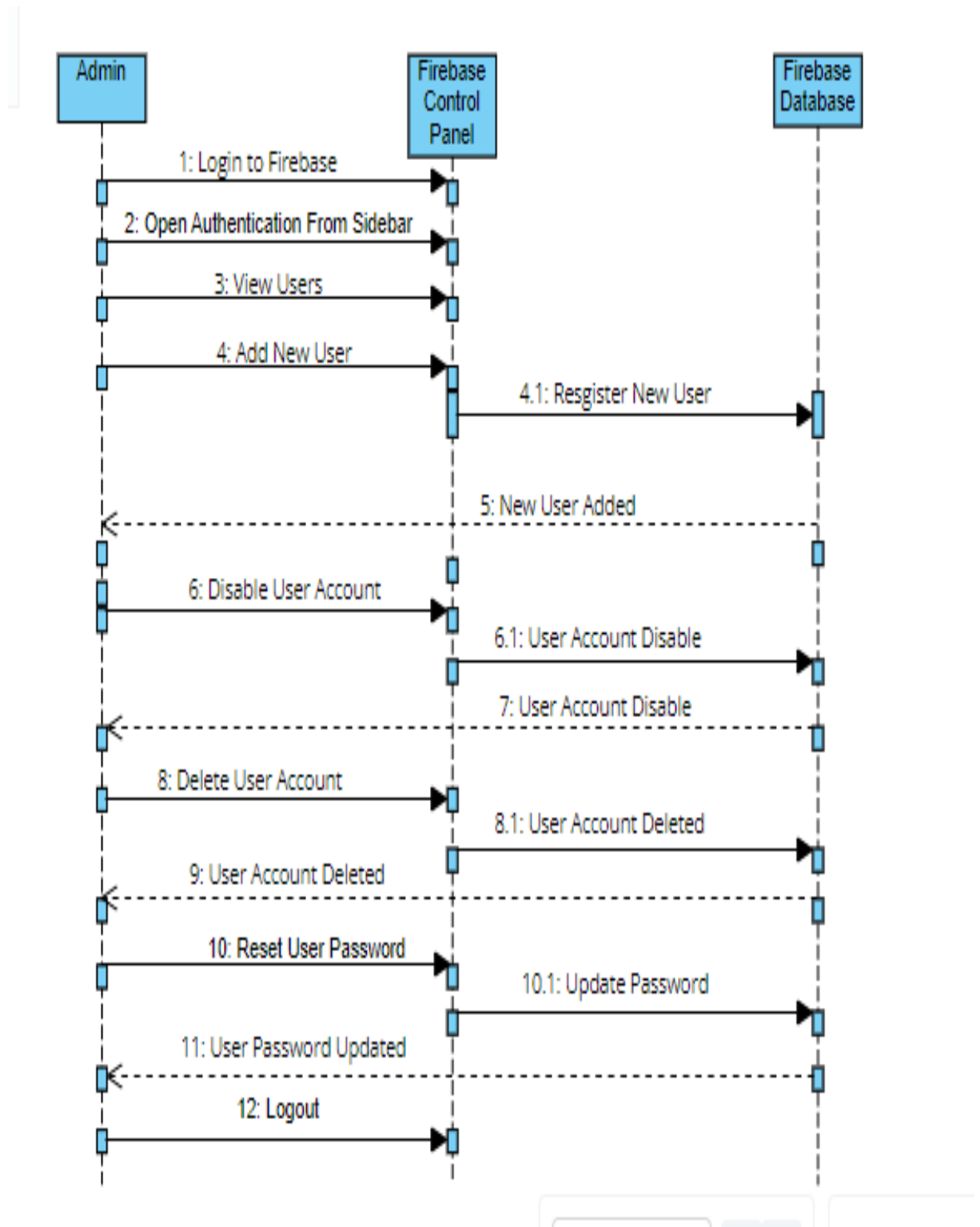


Figure: Admin Sequence Diagram

6.5 Activity Diagram

An activity diagram visually presents a series of actions or flow of control in a system similar to a flowchart or a data flow diagram. Activity diagrams are often used in business process modeling. They can also describe the steps in a use case diagram.

User Activity Diagram

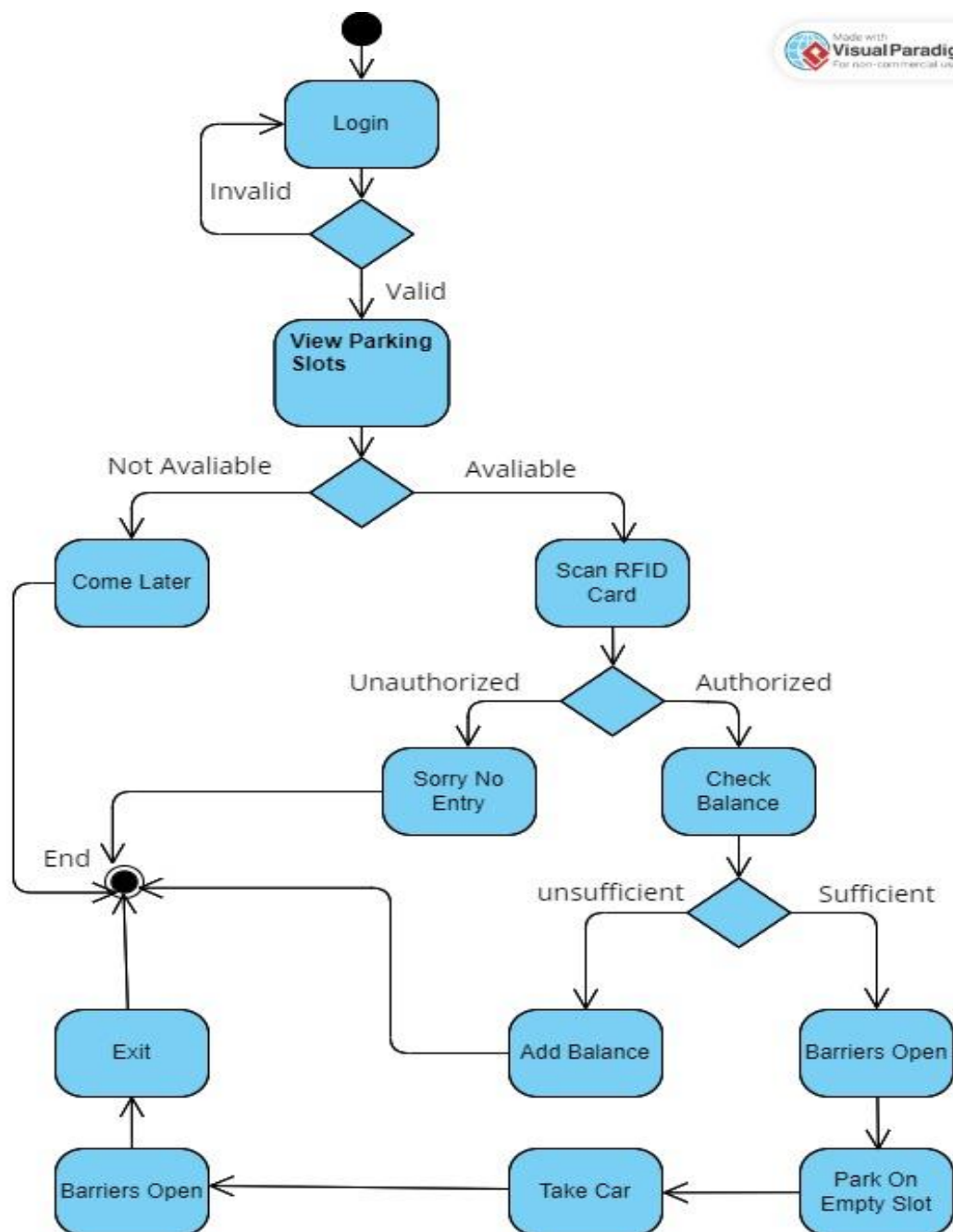


Figure: User activity Diagram

Admin Activity Diagram

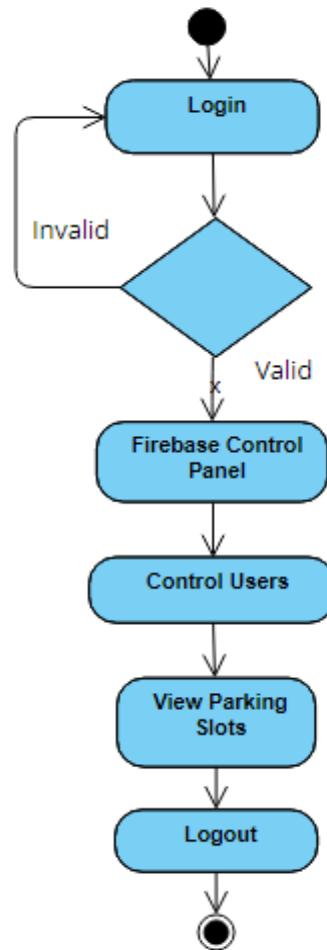


Figure: Admin activity Diagram

Implementation

Implementing the E-Parking system using an Android app involves developing the app with features like user authentication, real time configuration with data base and Local server connectivity for communication between sensors and App. It requires coding in Java, and Arduino IDE to connect software with hardware, and ensuring a user-friendly interface. Firebase is used to support main features of application like User No-SQL database and authentication and data storage.

7.1 Introduction

Following Development environment is used while developing Auction+:

- Android Studio
- Firebase
- Visual Studio Code
- Arduino IDE
- MS Project for project management
- MS Word for documentations
- Visual Paradigm for system design and diagrams

7.1.1 Languages:

User/Application End languages:

- Java
- C++ (Arduino)

7.1.2 Hardware Components

Following are the hardware components that we are going to use in our project:

Microcontroller (Arduino Uno)

Arduino Uno is a microcontroller board based on the ATmega328P chip. It is easy to use and program, making it a popular choice for beginners and hobbyists. Its benefits include low cost, versatility, and compatibility with a wide range of sensors and shields.

It can be programmed using the Arduino IDE and is widely used for DIY electronics projects. Its features include 14 digital input/output pins, 6 analog inputs, a 16 MHz quartz crystal, a USB connection, and a power jack.

Servo Motor

A servo motor is a rotary actuator that allows for precise control of angular or linear position, velocity, and acceleration. Its features include high torque, fast response time, and the ability to maintain position without feedback. Servo motors

are widely used in robotics, automation, and industrial applications for their precise and accurate control capabilities.

RFID Scanner

RFID (Radio-Frequency Identification) scanner is a device used to read and capture data stored on RFID tags or transponders. Its features include the ability to capture data wirelessly, real-time tracking and inventory management, and automatic identification and authentication of objects or people. RFID scanners are widely used in various applications such as access control, asset tracking.

WIFI Module

A Wi-Fi module is a device that enables wireless communication between electronic devices. Its features include high-speed data transfer rates, low power consumption, and compatibility with a wide range of devices. Wi-Fi modules are widely used in applications such as IoT, home automation, and wireless sensor

networks for their ability to provide wireless connectivity to devices.

IR Sensors

An IR sensor is an electronic device that detects infrared radiation and converts it into an electrical signal. Its features include high sensitivity, low power consumption, and the ability to detect motion and proximity. IR sensors are widely used in applications such as home automation, security systems, and industrial automation.

Wirings

Wire is a thin, flexible, and insulated metal conductor used for transmitting electrical signals or power between different components in electronic circuits

Wero Board

A wero board is a device used to prototype and test electronic circuits without the need for soldering. It allows electronic components such as resistors, capacitors, and integrated circuits to be easily connected together in a temporary manner. The holes on the breadboard are typically arranged in a grid, allowing for easy placement of components and connections using wires.

LED Display

A 20x4 LCD (Liquid Crystal Display) is a commonly used display module. It can display alphanumeric characters and symbols and is compatible with the Arduino microcontroller. It is often used for displaying real-time data such as sensor values, system status, or messages in various Arduino projects.

I2C

I2C (Inter-Integrated Circuit) is a protocol used to communicate between devices, allowing multiple devices to be connected to a single bus. In the context of an Arduino project using an LCD, I2C allows for the use of a serial bus to connect the LCD to the Arduino, using only two data pins, rather than the multiple pins required for a parallel connection. This reduces the number of pins used by the LCD, leaving more pins available for other components or functions.

7.2 Coding

After completion of design phase, implementation phase started. We are following incremental model, and each phase is completed in increments as per its requirements before moving to the next one and we can edit whenever we want.

Firebase is used as backbone of application; Auth library is used for user's authentication and verification. Firebase APIs of real-time database is used in the processes in application. All data is stored in firebase database provided by firebase.

Once a user is added to application by admin, user details are added in firebase, firebase receives the list of users registered to application and user can login with their created credentials, after that user will be on the main screen named as home screen. User can view all available parking slots (free and occupied) and can park on any slot after successful authorization using RFID Card. We are using dummy wallet for payment. After paying the amount vehicle will have access to parking zone. Firebase will update in few seconds for the real time database at the backend and Android Application provides good experience for this Project.

7.3 Verification

Verification of E-Parking is divided into two techniques one for UI testing and other for functionality testing. Verification of E-Parking system using an Android app is a critical step to ensure that the system functions and feature works as expected and performs its intended tasks accurately, correctly and meets its intended goals. Here is an outline of the verification process for such a system:

Functional Testing:

Test the core functionalities of the Android app, such showing empty and occupied slots. Ensure that each feature works as expected and performs its intended tasks accurately.

Performance Testing:

Assess the app's performance under various conditions, such as poor network connectivity, verify that it remains responsive and functional even in challenging environments.

Integration Testing:

If the E-Parking system includes hardware components ensure that the Android app integrates seamlessly with these components and communicates effectively.

Beta Testing

Beta testing involves users' feedback after using the application. Users report any sort of issues and errors found within the application. These issues can be related to application UI or application functionality. E-Parking beta version is tested among E-Parking team members and supervisor. In the beginning a number of issues were found both UI related as well as functionality related. Then after the feedback, those issues were resolved until complete user satisfaction.

Black Box Testing

Application features are tested using black box testing. Black box involves the user input/action and expected output/behavior of application. Using SRS and system design test cases are written for each possible user input and corresponding output. Each input was given to application and verified actual output for expected output.

7.3.1 *Test cases:*

7.3.1.1 SELECT LOGIN TYPE TEST CASE

Table 7.1: Select login type test case

Test Case No.	TC-1
Purpose of the Test	To verify login is successful or not
Input	Login with credentials (email, password)
Expected output	Show the message login successful
Pass/Fail	Pass

7.3.1.1 VERIFY WRONG CREDENTIALS RESPONSE TEST CASE

Table 7.2: Wrong credentials response test case

Test Case No.	TC-2
Purpose of the Test	To verify user is not available in system database
Input	Login with a wrong email that is not signed up yet
Expected output	Alert that shows; Authentication Failed
Pass/Fail	Pass

7.4 Validation

Every system has some validity criteria, so E-Parking also has some. For the validity of application module, every module is passed through validation testing. The modules for validation are highlighted in the application and a validity criterion is created according to the requirement of that specific module. E-Parking validation test is conducted between FYP team. All modules are validated and approved by the team.

CHAPTER 8

Discussion and Conclusion

8.1 Solution Review

We addressed the problem effectively. Our proposed solution is highly reliable, safe and secure. It provides an effective and easy way to automate the conventional parking system to automatic smart parking system. Moreover, the android app helps to save time that can be wasted in search of available parking slot. This type of parking system can be used in various places like societies parking, in shopping malls, etc.

8.2 Key Skills

Key skill learned are improved in this project's design and development are stated below:

- Android App Development
- Knowledge of IOT and IOT Devices
- Java
- C++ (Arduino Programming)
- Usage of Firebase
- Teamwork and Consistency

8.3 Future Work

This project was a great learning opportunity and it helped us to learn new technologies that have great demand in the industry nowadays. As for concerned with job, I thing this tech stack will be quite helpful to get a newbie job right after our completion of degree.

Moreover, this project will be in great demand in our country in future because the advancement in technology is affecting every field and transportation

sector is no exception.

8.4 Conclusion

This mobile application is useful as it will introduce a new and improved way of parking vehicle as compared to the traditional offline method. People will be able to park their vehicles without wasting their time using mobile app. The "Android Based E-parking system with RFID" project successfully demonstrates the convergence of modern mobile applications with real-world, hardware-based systems to address urban challenges. Leveraging the ubiquity of Android devices, this system provides users with an intuitive interface to interact with parking spaces, while the integration of RFID technology ensures secure, fast, and efficient vehicle identification and parking management.

REFERENCES

(Mainetti, Palano et al. 2014, Mahendra, Sonoli et al. 2017, Sadhukhan 2017, Cynthia, Priya et al. 2018, Al-Turjman and Malekloo 2019, Saeliw, Hualkasin et al. 2019, Elsonbaty and Shams 2020, Ilham, Yutanto et al. 2020, Mudjanarko, Kamisutara et al. 2020)

[1]Al-Turjman, F. and A. Malekloo (2019). "Smart parking in IoT-enabled cities: A survey." Sustainable Cities and Society **49**: 101608.

Cynthia, J., et al. (2018). "IOT based smart parking management system." International Journal of Recent Technology and Engineering (IJRTE) **7**(4S): 374-379.

Elsonbaty, A. and M. Shams (2020). "The smart parking management system." arXiv preprint arXiv:2009.13443.

Ilham, R., et al. (2020). "Analysis of the Acceptance Factor of Android-Based Parking Information Systems in Indonesia." Communications-Scientific Letters of the University of Zilina **22**(2): 97-106.

Mahendra, B., et al. (2017). IoT based sensor enabled smart car parking for advanced driver assistance system. 2017 2nd IEEE International Conference on Recent Trends in Electronics, Information & Communication Technology (RTEICT), IEEE.

Mainetti, L., et al. (2014). Integration of RFID and WSN technologies in a Smart Parking System. 2014 22nd international conference on software, telecommunications and computer networks (SoftCOM), IEEE.

Mudjanarko, S. W., et al. (2020). Android-based information technology design for easy parking. Proceedings of the International Conference on Industrial Engineering and Operations Management.

Sadhukhan, P. (2017). An IoT-based E-parking system for smart cities. 2017 International conference on advances in computing, communications and informatics (ICACCI), IEEE.

Saeliw, A., et al. (2019). "Smart car parking mobile application based on RFID and IoT."

More, Vipul & Ravariya, Kiran & Shah, Sohil & Solkar, Azharuddin. (2017). AUTOMATIC CAR PARKING SYSTEM USING RFID. 10.13140/RG.2.2.15282.12483.

P Joshi, M.R Khan and L Motiwalla, Global Review of Parking Management System and Strategies. Volume 2, Issue 6, June-2011 1 ISSN 2279-5141.

C.Patel, M.Swami, P.Saikia, S.Shah, Rotary Automated Car Parking system ,International Journal of Engineering Science and Innovative Technology (IJESIT) Volume 4, Issue 2, March 2015